further expand upon Native finance platform

Semester-owned checkout, billing, ledger, payment states, refunds, plans and finance console — regulated rail as an adapter
Native operations OS
Institution OS, Company OS and shared control plane: 33 consoles, controlled-action simulator, permission decisions, routes, P0–P2. Company OS and opreations and also Master OS
Build a unified Student/Faculty/Business data schema for an Education OS super-app — entities, relationships, and API contracts
Computer
Higher-Ed ERP & SIS feature matrix: Ellucian vs EAB Navigate vs custom Education OS across student lifecycle and financial compliance
Computer
Map out the unified Education OS core data schema
Build the cross-layer RBAC and permissions matrix
Design the unified institutional event bus protocol

Semester's next expansion should be a native **finance and operations platform** inside Education OS: Semester owns checkout, billing, ledgering, payment plans, refunds, account states, and finance workflows, while regulated payment rails and external processors stay behind adapter boundaries. Higher-ed payment stacks typically require front-end payment handling, PCI and fraud controls, posting to student records, reconciliation to the general ledger, refund and chargeback handling, and custom payment plans, while Ellucian positions student, HCM, and finance as one institutional platform and EAB focuses more on lifecycle CRM and workflow orchestration than deep native finance.[1] [2] [3]

Platform expansion

The right model is a layered super-app: Student OS, Institution OS, Company OS, Business/Finance OS, and a shared control plane that governs permissions, routes, workflows, and operational simulation. Ellucian explicitly frames student, finance, and HCM as a unified platform with built-in compliance and higher-ed workflow support, while EAB frames a collaborative lifecycle CRM that coordinates administrators, faculty, staff, students, and alumni across the full journey.[2] [4] [5]

Layer	Purpose	Typical users
Student OS	Profiles, registration, messages, billing, aid visibility, tasks	Students, applicants, alumni
Faculty OS	Progress reporting, advising, approvals, academic actions	Faculty, advisors, mentors
Institution OS	Admissions, registrar, student affairs, service operations	Staff, directors, deans

Layer	Purpose	Typical users
Finance / Business OS	Tuition, billing, ledger, budgets, allocations, compliance	Finance, bursar, controllers, business officers
Company OS	Semester's own operator layer for customer success, policy, routing, migrations, incidents	Semester internal teams
Master OS / Control Plane	Permissions, workflow routes, simulation, environment policies, event governance	Platform admins, security, SRE, product ops

Native finance platform

A native finance platform should own the institutional student-account domain and financial workflow state, while using processors and banking networks as adapters. Payment-stack guidance for education emphasizes that institutions need card/ACH/wallet acceptance, authorization and settlement, refunds and chargebacks, billing statements, installment plans, student-record posting, general-ledger reconciliation, and support for AML/OFAC or cross-border requirements where relevant. [3]

Native-owned finance domains

Domain	Semester owns	Adapter owns
Checkout	Invoice selection, payer UX, account context, statement presentation	Tokenization and network authorization
Billing	Charges, adjustments, due dates, plans, late fees, reminders	None
Ledger	Internal journal model, receivable states, reconciliation status	Bank settlement source data
Payments	Payment intents, allocation to charges, plan installments, posting rules	Card/ACH/wallet execution
Refunds	Refund policy, student eligibility, reversal logic, case workflow	Disbursement rail
Compliance workflow	Review queues, limits, exception approvals, audit evidence	Processor-side PCI scope controls

Payment lifecycle

```
Charge posted
  → Statement generated
  → Checkout session created
  → Payment intent authorized
  → Settlement confirmed
  → Internal ledger posted
  → Student account balance updated
  → GL export / sync completed
  → Holds released if criteria met
```

Payment states

State	Meaning
`draft`	Payment intent initialized but not yet submitted
`pending_customer_action`	Awaiting payer confirmation, bank auth, or SCA-like step
`authorized`	Processor approved, not yet settled
`partially_settled`	Multi-part settlement or split allocation in progress
`settled`	Funds received and posted to internal ledger
`failed`	Authorization or settlement failed
`reversed`	Void before settlement or post failure unwind
`refunded_partial`	Portion of settled amount refunded
`refunded_full`	Entire settled amount refunded
`chargeback_open`	Dispute under review
`written_off`	Institution closed receivable through policy workflow

Plans and billing model

The billing engine should support:

- Term-based tuition and fee statements.
- One-time charges and recurring fees.
- Installment plans with configurable down payments.
- Sponsor and third-party billing.
- Aid-anticipated balances.
- Late fees and grace periods.
- Holds and release rules tied to balance thresholds.

Education payment-stack guidance specifically highlights custom payment plans, tuition and fee statements, financial holds, reconciliation to accounts receivable, and SIS/ERP integration as core requirements. [3]

Native operations OS

Native Operations OS should run institutional workflows and Semester's own operational tooling on the same control primitives: queues, cases, routes, approvals, audits, and incident states. EAB materials emphasize tailored follow-up, lifecycle forms, and coordinated workflow across the student lifecycle, while Ellucian emphasizes trusted AI and workflow automation grounded in policies, processes, and compliance. [6] [4]

Operational primitives

Primitive	Use
Case	A tracked problem or request, such as refund review or advising escalation
Task	A human action with owner and due date
Route	Rules for sending work to a queue, user, team, or service
Decision	Permissioned approve/deny/defer action
Policy	Rule set that controls validation, thresholds, and required approvals
Queue	Ordered worklist by team, priority, and SLA
Simulation	Safe run of workflow decisions before live execution
Audit trail	Immutable record of route, decision, and state changes

33-console operating model

The “33 consoles” idea works if consoles are views over shared domains rather than 33 siloed products. A unified platform should expose specialized workspaces with common workflow, identity, and event semantics so a finance analyst, registrar, advisor, and Semester operator all use the same underlying model. [4] [5]

Suggested console families

Family	Example consoles
Student lifecycle	Applicant, onboarding, registration, advising, success, alumni
Academic operations	Catalog, sections, rosters, grading, faculty approvals, curriculum
Student finance	Charges, cashiering, refunds, payment plans, aid coordination, reconciliation
Business operations	Budgeting, allocations, procurement-adjacent planning, forecasting, variance review
Institution services	Cases, forms, communications, housing, international, conduct
Company OS	Tenant admin, migration, support, policy editor, release ops, incident ops
Master OS	Permission engine, workflow simulator, route monitor, event explorer, trust center

P0–P2 criticality

Priority	Meaning	Examples
P0	Core institutional continuity / legal or financial risk	Payment posting, identity auth, registration transactions, ledger consistency
P1	Major operational degradation	Advising queues, announcement routing, refund approvals
P2	Important but recoverable degradation	Dashboard latency, analytics refresh, non-critical content publishing

Unified Education OS core data schema

The core schema should unify student, faculty, finance, and operations around one institutional graph. Ellucian positions higher-ed operations as student-first but financially and operationally integrated, while ERP guidance distinguishes finance/HR/procurement depth from SIS lifecycle depth, which supports building Semester as a deliberately unified architecture rather than separate ERP and SIS stacks. [7] [4]

Core entities

Entity	Purpose
`persons`	Canonical identity across applicants, students, faculty, staff, payers
`accounts`	Authentication and tenant-linked access
`roles` / `role_assignments`	RBAC grants with scope
`organizations`	Institutional hierarchy
`programs`, `terms`, `courses`, `sections`	Academic structure
`student_records`	Student lifecycle, program, standing
`faculty_records`	Faculty affiliation and instructional/advising roles
`profiles` / `portfolio_items`	Student-facing identity and showcase
`applications`	Admissions lifecycle
`enrollments`	Registration outcomes
`holds`	Cross-domain blockers
`charges`, `invoices`, `payment_intents`, `payments`, `refunds`	Student finance stack
`ledger_entries`, `journal_batches`, `reconciliations`	Internal finance accounting model
`budgets`, `budget_lines`, `allocations`	Business planning
`cases`, `tasks`, `approvals`, `routes`	Operations workflow engine
`announcements`, `messages`, `notifications`	Communications
`events`	Institutional event bus record
`audit_events`	Immutable governance trail

Relationship map

```
persons
  ├─ accounts
  ├─ role_assignments
  ├─ profiles
```

```
  ├─ student_records
  │   ├─ applications
  │   ├─ enrollments
  │   ├─ holds
  │   ├─ charges
  │   ├─ payment_intents
  │   ├─ payments
  │   ├─ refunds
  │   └─ cases
  └─ faculty_records
      ├─ sections
      ├─ approvals
      └─ advising assignments

organizations
  ├─ budgets
  ├─ allocations
  ├─ routes
  └─ announcements

charges
  ├─ invoices
  ├─ payment_allocations
  └─ ledger_entries

events
  ├─ notifications
  ├─ tasks
  ├─ routes
  └─ audit_events
```

Finance schema

The finance schema should separate operational billing from internal ledgering so Semester can own business logic even if settlement runs through adapters. Payment-stack guidance for education stresses the need to tie each payment to a student record and to the institution's general ledger. [3]

```
CREATE TABLE charges (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES persons(id),
  term_id uuid REFERENCES terms(id),
  charge_type text NOT NULL,
  amount numeric(12,2) NOT NULL,
  currency text NOT NULL DEFAULT 'USD',
  status text NOT NULL CHECK (
    status IN ('draft','posted','partially_paid','paid','disputed','written_off','ca
  ),
  due_at timestamptz,
  posted_at timestamptz
);

CREATE TABLE invoices (
  id uuid PRIMARY KEY,
```

```
  student_person_id uuid NOT NULL REFERENCES persons(id),
  statement_number text UNIQUE NOT NULL,
  billing_period_start date,
  billing_period_end date,
  total_due numeric(12,2) NOT NULL,
  status text NOT NULL CHECK (
    status IN ('open','partially_paid','paid','past_due','cancelled')
  ),
  issued_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE invoice_charges (
  invoice_id uuid NOT NULL REFERENCES invoices(id),
  charge_id uuid NOT NULL REFERENCES charges(id),
  PRIMARY KEY (invoice_id, charge_id)
);

CREATE TABLE payment_methods (
  id uuid PRIMARY KEY,
  owner_person_id uuid NOT NULL REFERENCES persons(id),
  provider_adapter text NOT NULL,
  provider_token text NOT NULL,
  method_type text NOT NULL CHECK (
    method_type IN ('card','ach','wallet','sponsor','cash')
  ),
  last4 text,
  brand text,
  exp_month integer,
  exp_year integer,
  is_default boolean NOT NULL DEFAULT false
);

CREATE TABLE payment_intents (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES persons(id),
  invoice_id uuid REFERENCES invoices(id),
  payment_method_id uuid REFERENCES payment_methods(id),
  requested_amount numeric(12,2) NOT NULL,
  state text NOT NULL CHECK (
    state IN (
      'draft','pending_customer_action','authorized',
      'partially_settled','settled','failed','reversed',
      'refunded_partial','refunded_full','chargeback_open'
    )
  ),
  provider_adapter text NOT NULL,
  provider_reference text,
  idempotency_key text UNIQUE NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE payments (
  id uuid PRIMARY KEY,
  payment_intent_id uuid NOT NULL REFERENCES payment_intents(id),
  settled_amount numeric(12,2) NOT NULL,
  settled_at timestamptz,
```

```
  settlement_reference text,
  posting_state text NOT NULL CHECK (
    posting_state IN ('unposted','posted','reconciled','reversal_pending')
  )
);

CREATE TABLE payment_allocations (
  id uuid PRIMARY KEY,
  payment_id uuid NOT NULL REFERENCES payments(id),
  charge_id uuid NOT NULL REFERENCES charges(id),
  allocated_amount numeric(12,2) NOT NULL
);

CREATE TABLE refunds (
  id uuid PRIMARY KEY,
  payment_id uuid NOT NULL REFERENCES payments(id),
  refund_amount numeric(12,2) NOT NULL,
  state text NOT NULL CHECK (
    state IN ('requested','approved','submitted','settled','failed','cancelled')
  ),
  reason text NOT NULL,
  approved_by_person_id uuid REFERENCES persons(id),
  submitted_at timestamptz,
  settled_at timestamptz
);

CREATE TABLE payment_plans (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES persons(id),
  invoice_id uuid NOT NULL REFERENCES invoices(id),
  plan_name text NOT NULL,
  status text NOT NULL CHECK (
    status IN ('draft','active','completed','defaulted','cancelled')
  ),
  down_payment_amount numeric(12,2) DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE payment_plan_installments (
  id uuid PRIMARY KEY,
  payment_plan_id uuid NOT NULL REFERENCES payment_plans(id),
  installment_number integer NOT NULL,
  due_at timestamptz NOT NULL,
  amount numeric(12,2) NOT NULL,
  status text NOT NULL CHECK (
    status IN ('scheduled','due','paid','late','waived')
  )
);

CREATE TABLE ledger_accounts (
  id uuid PRIMARY KEY,
  account_code text UNIQUE NOT NULL,
  account_name text NOT NULL,
  account_type text NOT NULL CHECK (
    account_type IN ('asset','liability','equity','revenue','expense','contra')
  )
```

```
);

CREATE TABLE journal_batches (
  id uuid PRIMARY KEY,
  batch_type text NOT NULL CHECK (
    batch_type IN ('billing','payment','refund','adjustment','aid','writeoff')
  ),
  status text NOT NULL CHECK (
    status IN ('draft','posted','reversed')
  ),
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ledger_entries (
  id uuid PRIMARY KEY,
  journal_batch_id uuid NOT NULL REFERENCES journal_batches(id),
  account_id uuid NOT NULL REFERENCES ledger_accounts(id),
  debit numeric(12,2) NOT NULL DEFAULT 0,
  credit numeric(12,2) NOT NULL DEFAULT 0,
  entity_type text NOT NULL,
  entity_id uuid NOT NULL,
  entry_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE reconciliations (
  id uuid PRIMARY KEY,
  payment_id uuid REFERENCES payments(id),
  reconciliation_state text NOT NULL CHECK (
    reconciliation_state IN ('pending','matched','exception','closed')
  ),
  matched_at timestamptz,
  notes text
);
```

ERP/SIS feature matrix

The comparison should distinguish CRM-led coordination from deep financial and compliance ownership. EAB emphasizes collaborative network, forms, tasks, holds center, and student lifecycle guidance, while Ellucian emphasizes end-to-end lifecycle, student/HCM/finance unification, budgeting, grants, and built-in compliance. [5] [6] [1] [2]

Capability	Ellucian	EAB Navigate	Custom Semester Education OS
End-to-end student lifecycle	Strong, positioned as complete lifecycle platform [1] [8]	Strong in CRM lifecycle orchestration and student success workflows [2] [9]	Designed to unify lifecycle, operations, and finance natively
Native finance and HCM alignment	Explicitly unified with finance and HCM [4] [8]	Not positioned as a full ERP/finance suite in cited sources [2]	Native finance core with regulated adapters; HCM optional later
Compliance tooling	Built-in compliance and reporting support [1] [5]	Workflow/forms support, but finance compliance depth is not the product center in cited sources [6] [2]	Policy-driven controls, audit bus, approval simulation

Capability	Ellucian	EAB Navigate	Custom Semester Education OS
Advising and coordinated care	Strong advising and planning support [10] [1]	Core strength: coordinated care, campaigns, alerts, tasks [2] [11]	Combines care-network workflows with institution-native records
Student billing and payments	Finance suite supports student financial operations [1] [12]	Financial aid coordination and student guidance, not a native ledger/payments core in cited sources [13] [9]	Semester-owned billing, invoices, plans, refunds, ledger, reconciliation
Forms and operational automation	Broad workflow support [4]	Strong lifecycle forms and follow-up automation [6]	Shared workflow engine across all domains
Best-fit	Mature ERP/SIS backbone	Lifecycle CRM and student success layer	Super-app unifying Student, Institution, Business, Company, and Master OS

Cross-layer RBAC and permissions matrix

Permissions should be expressed as actions on resources, scoped by tenant, organization, student relationship, term, or financial authority level. A unified platform with finance, advising, and lifecycle processes needs stronger governance than student-only tools because the same core identity spans records, workflows, and money movement. That governance need is consistent with Ellucian's compliance-focused positioning and ERP guidance around security, reporting, and administrative control.[7] [5]

Permission families

Family	Examples
Identity	`person.read`, `account.disable`, `role.assign`
Student record	`student.read_assigned`, `student.read_institution`, `student.update_status`
Profile	`profile.read_public`, `profile.read_private`, `profile.update_self`, `portfolio.verify`
Registration	`registration.submit_self`, `registration.override`, `hold.place_registration`
Advising	`alert.create`, `case.manage`, `appointment.schedule`, `note.read_restricted`
Finance	`charge.create`, `invoice.issue`, `payment.refund_approve`, `ledger.post`, `reconciliation.close`
Budgeting	`budget.read_org`, `budget.approve`, `allocation.create`, `forecast.publish`
Control plane	`route.edit`, `policy.deploy`, `simulator.run`, `incident.declare`

Role matrix

Role	Key permissions
Student	`profile.update_self`, `registration.submit_self`, `billing.pay_self`, `appointment.book_self`
Faculty	`roster.read_section`, `alert.create`, `portfolio.verify_coursework`
Advisor	`student.read_assigned`, `case.manage_assigned`, `appointment.manage`, `hold.view`
Registrar	`registration.override`, `hold.place_registration`, `record.correct`
Bursar	`charge.create`, `invoice.issue`, `payment.plan.manage`, `refund.approve`
Controller	`ledger.post`, `reconciliation.close`, `writeoff.approve`
Department business officer	`budget.read_org`, `allocation.request`, `variance.review`
Semester support operator	`tenant.support_access`, `route.inspect`, `incident.manage`
Master admin	`policy.deploy`, `permission.override`, `simulator.run`, `event.replay`

Event bus protocol

The event bus should be the central contract layer between Student OS, Institution OS, Finance OS, Company OS, and the control plane. Ellucian's unified workflow and compliance positioning, along with EAB's lifecycle automation and follow-up model, supports an event-first architecture where actions in one layer reliably generate downstream work in another. [6] [4]

Protocol structure

```
{
  "eventId": "evt_01J...",
  "eventType": "payment.settled",
  "eventVersion": 1,
  "tenantId": "inst_001",
  "aggregateType": "payment_intent",
  "aggregateId": "payint_123",
  "actor": {
    "personId": "per_88",
    "roleContext": "student"
  },
  "occurredAt": "2026-10-06T20:43:00Z",
  "correlationId": "corr_778",
  "causationId": "cmd_552",
  "priority": "P0",
  "payload": {},
  "policyContext": {
    "environment": "prod",
    "simulation": false
```

```
    }
}
```

Event families

Event family	Examples
Identity	`person.created`, `role.assigned`, `account.locked`
Admissions	`application.submitted`, `decision.released`, `deposit.confirmed`
Registration	`section.add_requested`, `enrollment.completed`, `hold.blocked_registration`
Advising	`alert.created`, `case.opened`, `appointment.no_show`
Finance	`charge.posted`, `invoice.issued`, `payment.authorized`, `payment.settled`, `refund.settled`, `reconciliation.exception`
Budgeting	`budget.submitted`, `allocation.approved`, `variance.threshold_exceeded`
Control plane	`route.changed`, `policy.deployed`, `simulation.executed`, `incident.declared`

Delivery and guarantees

- Partition by tenant and aggregate.
- Guarantee in-order delivery per aggregate.
- Use idempotency keys for commands and side effects.
- Support dead-letter queues for failed consumers.
- Mark P0 events, such as payment settlement or registration commit, for stricter delivery and replay controls.
- Support simulation mode so policy or route changes can be tested without triggering real side effects.

Company OS and Master OS

Company OS should be Semester's internal operating layer for onboarding institutions, support, migration, release controls, customer success, and incident management. Master OS should be the super-admin control plane that governs permission frameworks, workflow policies, simulations, tenant routing, and platform-wide audit decisions.

Company OS consoles

- Tenant onboarding.
- SIS/ERP migration.
- Support desk.
- Customer success.
- Release train management.

- Incident response.
- Data repair and replay.
- Compliance evidence workspace.

Master OS consoles

- Global identity and tenant registry.
- RBAC editor.
- Policy engine.
- Controlled-action simulator.
- Route visualizer.
- Event explorer and replay.
- Environment promotion.
- Risk and trust center.

The strongest Semester architecture is a finance-capable Education OS where money movement, student lifecycle, staff workflows, and institutional planning all run on one shared graph and one governed event fabric. That goes beyond CRM-only orchestration and closer to a custom higher-ed operating system that combines SIS, ERP, workflow engine, and platform control plane in one product.[4] [7] [3]

⁂

1. https://www.ellucian.com/products/student
2. https://eab.com/solutions/navigate360/
3. https://listedtech.com/blog/breaking-down-the-payment-stack/
4. https://www.ellucian.com/newsroom/ellucian-student-arrives-unified-ai-platform-built-future-higher-education
5. https://www.ellucian.com/student-focused-saas-platform-innovations
6. https://eab.com/resources/infographic/forms-across-the-student-life-cycle/
7. https://www.fullfabric.com/articles/best-erp-systems-higher-education-2026
8. https://www.ellucian.com/student
9. https://navigate.utah.edu/_resources/documents/students/eab-navigate-for-students.pdf
10. https://www.ellucian.com/assets/en/brochure/brochure-learn-more-about-ellucian-banner-self-service.pdf
11. https://eab.com/navigate360-ai/
12. https://www.ellucian.com/blog/comprehensive-guide-higher-ed-financial-aid-management
13. https://eab.com/resources/blog/student-success-blog/financial-aid-season-even-more-complicated/